PROJECT REPORT

ON

JAMGUARD

Smart Motor Monitoring and Automatic Protection System

An Academic Project Report

SUBMITTED BY :

26BEC0268 – Patnala Subhaom Kumar Patro

26BEC0187 – Ruchir Nilesh Patil

26BEC0169 – Aarav Ghiya

26BEC0387 – Atharv Saxena

B.Tech First Year · Department of SENSE · Academic Year 2026–27

Guided by - Dr. KVN Kavitha

ABSTRACT

JamGuard is an educational prototype for monitoring a DC motor and providing automatic protection against sustained loss of rotation. An Arduino Uno R3 reads magnetic pulses from a Hall-effect sensor and a 5 mm magnet on the motor to derive RPM, while an L298N driver handles motor current. After a 3-second startup grace period, a 1.5-second no-pulse timeout (prototype settings) triggers motor disable and LED and buzzer alerts; a reset push-button can clear the protection state after inspection.

An ESP32 receives status from the Uno over UART and hosts a local Wi-Fi dashboard for RPM, motor status, and fault information. RPM = pulses per second × 60 / pulses per revolution; the 480 RPM example is illustrative, not a measured result. Uno-to-ESP32 serial requires 5V-to-3.3V level shifting or a voltage divider. The work integrates magnetic sensing, embedded control, motor protection, and local monitoring, with future scope for filtering, event logging, richer diagnostics, and validation.

Keywords: Arduino Uno R3, Hall-effect sensor, DC motor, RPM, L298N, ESP32, UART, Wi-Fi dashboard

1. INTRODUCTION

Motor rotation monitoring is important in conveyors, packaging machines, robotics, and automated gates. A motor may remain electrically energized while its shaft stalls, jams, or loses effective rotation. JamGuard extends basic motor control into a protection system by sensing magnetic pulses from a 5 mm magnet attached to a DC motor.

A Hall-effect sensor detects each passing magnet and the Arduino Uno R3 derives RPM from the pulse stream. The Uno applies a prototype startup grace period of 3 seconds and treats 1.5 seconds without a pulse as a fault condition.

It then disables the motor and activates an LED and buzzer, while an ESP32 receives status over UART and hosts a local Wi-Fi dashboard.

2. PROBLEM STATEMENT

A basic motor controller can switch the motor but may not detect sustained loss of rotation.

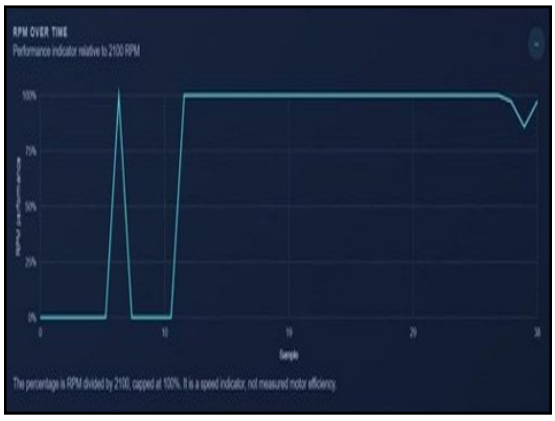
A low-cost system is therefore required to sense motor motion, calculate RPM, recognise a no-pulse condition after startup, stop the motor, and communicate clear status and fault information to a local dashboard.

3. AIM

To develop a low-cost smart motor monitoring and automatic protection prototype using an Arduino Uno R3, Hall-effect sensing, an L298N driver, and an ESP32 dashboard.

4. OBJECTIVES

- Interface a Hall-effect sensor and 5mm magnet with an Arduino Uno R3 module, which is a microcontroller.
- Derive motor RPM from magnetic pulse frequency. Drive and disable a DC motor through an L298N motor driver.
- Detect sustained loss of rotation after a prototype startup grace period of 3 sec.
- Use a 1.5-second prototype no-pulse timeout to trigger protection.
- Activate an LED and buzzer and support reset through a push-button.
- Transmit motor status over UART to an ESP32 using proper 5V-to-3.3V level shifting or a voltage divider.
- Display RPM, motor status, and fault information on a local Wi-Fi dashboard.



5. SYSTEM REQUIREMENTS

COMPONENTS	PURPOSE
ESP32 Dev Module	Monitoring of the motors, protection logic and sensor control, UART protocol status receiver, local Wi-Fi
Hall-effect sensor	Detects magnetic pulses from the rotating motor
5 mm magnet	Produces one magnetic event per revolution reference
DC motor	Motor under monitoring and control
L298N motor driver	Handles motor current and switching
Level shifting resistors	5V-to-3.3V divider for Uno-to-ESP32 UART

6. SOFTWARES USED

- Arduino IDE – programming and uploading firmware to the Arduino Uno R3 and ESP32.
- ESP32 local web dashboard – displays RPM, motor status, and fault information over Wi-Fi.

7. STEPS OF PROJECT

- Hall Sensor + Magnet affixed to Fan
- Arduino Uno R3 + Motor Control
- Protection Logic: RPM and No-Pulse Detection
- LED + Buzzer + DC 9V Motor
- Disable UART → ESP32 Local Wi-Fi Dashboard

8. PROPOSED CONNECTIONS

JamGuard Signal	Connection / Note
VCC	5V supply from the prototype circuit
GND	GND
TRIG	GPIO 5
ECHO	Use a proper 5V-to-3.3V level shifter or voltage divider

Motor driver inputs, LED, buzzer, reset push-button, and UART pins must be assigned in firmware; share a common ground, and protect every ESP32 input from the Uno's 5V logic.

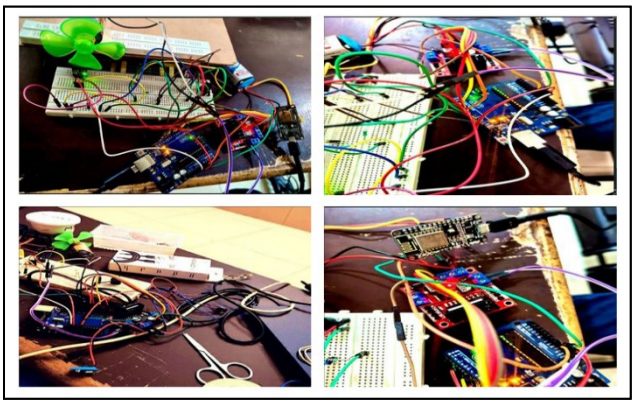
9. WORKING PRINCIPLE

A 5mm magnet attached to the DC motor passes a Hall-effect sensor when its TRIG pin is activated. The pulse travels through the air, reflects from an obstacle, and returns to the sensor. The ECHO signal remains active for a duration related to the round-trip travel time. The ESP32 measures this duration and calculates distance using the approximate speed of sound.

RPM = pulses per second × 60 / pulses per revolution

The RPM equation relates the pulse rate to motor speed; an illustrative value such as 480 RPM is not a measured prototype result. The Uno allows a 3-second startup grace period obstacle and back. In the program, a commonly used approximation is 0.0343 cm/µs for sound speed in typical conditions.

10. PROJECT PICTURES



11. PROJECT METHODS

STEPS	ABOUT THE STEP
Step 1– ESP32 setup	Configure the Arduino Uno R3 in Arduino IDE and verify pulse input and motor control.
Step 2 – Sensor connection	Mount the Hall-effect sensor and 5 mm magnet, and verify a pulse on each rotation.
Step 3 – Distance test	Count magnetic pulses and calculate RPM using pulses per second and pulses per revolution.
Step 4 – Servo integration	Apply the 3-second startup grace period and 1.5 second prototype no-pulse timeout.
Step 5 – Data transmission	Disable the motor and activate the LED and buzzer when a sustained fault is detected.
Step 6 – Serial monitoring	Send motor status, RPM, and fault information from the Uno to the ESP32 through properly level-shifted UART.
Step 7 – Coordinate changes	Host a local Wi-Fi page on the ESP32 showing RPM, motor status, and fault information.
Step 8 – Visualization	Use the reset push-button after inspection and review the dashboard state.

12. LIMITATIONS

- Hall-effect pulse quality can be affected by magnet alignment, sensor spacing, vibration, and electrical noise.
- The 5mm magnet and Hall sensor require stable mechanical mounting and a suitable sensing gap.
- RPM accuracy depends on pulse counting interval and the assumed pulses-per-revolution value.
- A no-pulse timeout may not distinguish a true jam from a disconnected sensor without additional diagnostics.
- The prototype protects one monitored motor and does not by itself model the full machine state.
- The prototype settings are not measured performance results; deployment requires validation, enclosure, and safety review.

13. FUTURE SCOPE

- Use filtering, debouncing, or multiple measurements to improve pulse stability.
- Add adaptive monitoring intervals and richer fault diagnostics for changing motor loads.
- Compare consecutive operating intervals to detect degradation or changing motor behavior.
- Add data logging for later maintenance analysis. Develop a clearer real-time ESP32 dashboard with event history and reset guidance.
- Extend the concept to multi-motor monitoring and machine-level status aggregation.
- Evaluate industrial-grade sensors and isolated interfaces for applications requiring higher reliability.

14. CONCLUSION

JamGuard demonstrates how magnetic pulse sensing can be combined with embedded control, motor driving, serial communication, and local web visualisation to form a smart motor monitoring and protection prototype.

The Arduino Uno R3 derives RPM from a Hall-effect sensor and 5 mm magnet, detects sustained loss of rotation using prototype timing settings, disables the motor, and activates an LED and buzzer.

The ESP32 receives status over properly level-shifted UART and presents RPM, motor status, and fault information on a local Wi-Fi dashboard.

The project provides practical experience in electronics and programming while establishing a foundation for future validation in conveyors, packaging machines, robotics, and automated gates.